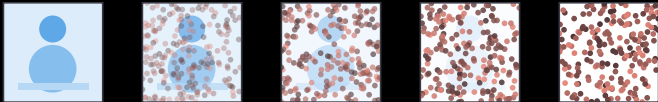
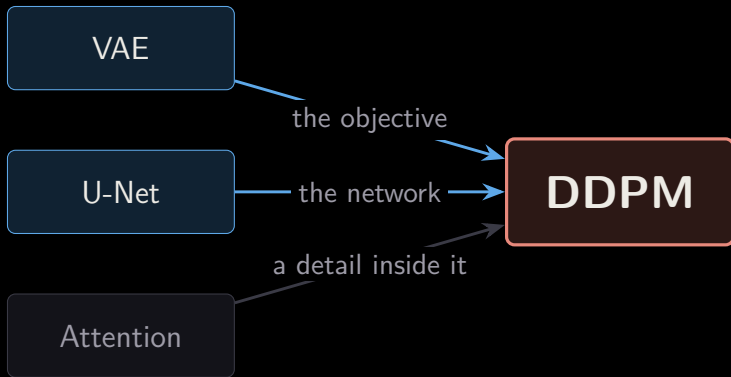


Denoising Diffusion Probabilistic Models

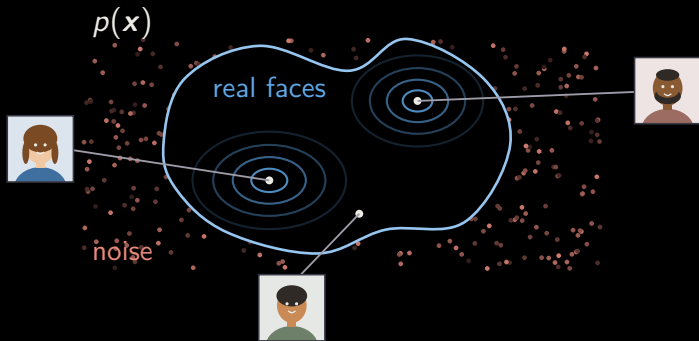


Ho, Jain & Abbeel · NeurIPS 2020 · [arXiv:2006.11239](https://arxiv.org/abs/2006.11239)

Built on things we've already done



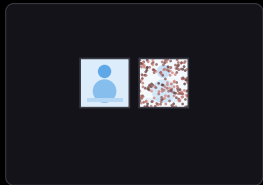
Where the real faces live



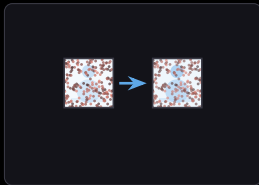
Almost every possible picture is static. Real images sit on a tiny island.

Brighter blue means more likely. We start in the noise and land on the island.

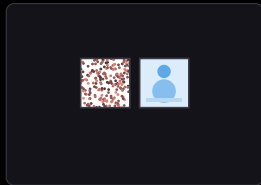
The idea



1. destroy



2. learn one undo



3. repeat 1000×

Generation = iterative denoising.

Two processes

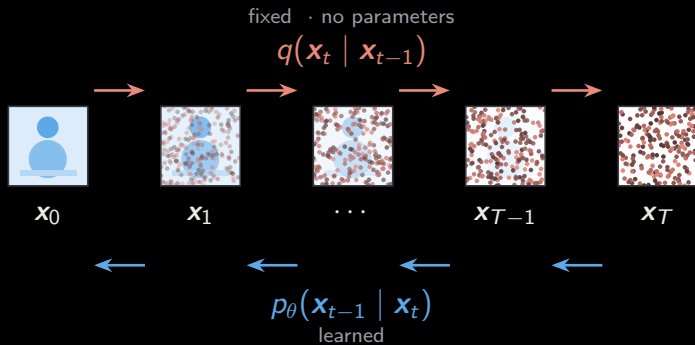


Figure 2

It's a VAE with the encoder removed

	VAE	Diffusion
encoder	learned	fixed noise
steps	1	1000
latent size	smaller	same as image
decoder	learned	learned
objective	ELBO	the same ELBO

The hard part of a VAE, learning an encoder, just disappears.

What is β_t ?



$$\beta_t = 0$$

nothing happens



$$\beta_t = 0.02$$

barely anything



$$\beta_t = 0.5$$

half gone



$$\beta_t = 1$$

all gone

the paper only ever uses this end

One knob: what fraction of the picture to replace with noise, this step.

Tiny steps are the price of reversibility. A big jump cannot be undone by a Gaussian.

The obvious way to add noise



Every step is $\mathcal{N}(x_{t-1}, \beta \mathbf{I})$, and n of them stack up to $\mathcal{N}(x_0, n\beta \mathbf{I})$.

the variance grows without bound

At $n = 1000$, $\beta = 0.02$: variance 20, so pixels swing by ± 4.5 .

Images only live in $[-1, 1]$.

So they fixed the variance

the obvious way

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_{t-1}, \beta \mathbf{I})$$

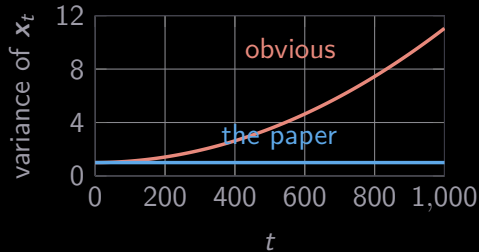
$$\mathbf{x}_t = \mathbf{x}_{t-1} + \sqrt{\beta} \epsilon$$

what the paper does

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1 - \beta} \mathbf{x}_{t-1}, \beta \mathbf{I})$$

$$\mathbf{x}_t = \sqrt{1 - \beta} \mathbf{x}_{t-1} + \sqrt{\beta} \epsilon$$

Same β in both. Only the mean of the distribution changed.



Forward: one step

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

shrink

so variance stays 1

add noise

variance $\beta_t \leq 0.02$

Do it 1000 times and you land at $\mathcal{N}(\mathbf{0}, \mathbf{I})$, which is free to sample.

Eq. 2

Skip straight to any step

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s \leq t} \alpha_s$$

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$
$$\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

A crossfade between image and noise.

We chose ϵ , so we already know the answer. **That's the label.**

Eq. 4

Where that came from: two steps at once

$$\begin{aligned}\mathbf{x}_2 &= \sqrt{\alpha_2} \mathbf{x}_1 + \sqrt{1 - \alpha_2} \epsilon_2 \\ &= \sqrt{\alpha_2} \left(\sqrt{\alpha_1} \mathbf{x}_0 + \sqrt{1 - \alpha_1} \epsilon_1 \right) + \sqrt{1 - \alpha_2} \epsilon_2 \\ &= \sqrt{\alpha_1 \alpha_2} \mathbf{x}_0 + \sqrt{\alpha_2 (1 - \alpha_1)} \epsilon_1 + \sqrt{1 - \alpha_2} \epsilon_2 \\ &= \sqrt{\alpha_1 \alpha_2} \mathbf{x}_0 + \sqrt{1 - \alpha_1 \alpha_2} \epsilon\end{aligned}$$

Two Gaussians add, so variances add: $\alpha_2(1 - \alpha_1) + (1 - \alpha_2) = 1 - \alpha_1\alpha_2$.

Same shape as one step, with $\alpha_1\alpha_2$ in place of α . After t steps that product is $\bar{\alpha}_t$.

Backwards: the hard direction

$$p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})$$

Gaussian is a fair guess *because* each step was tiny.

σ_t^2 is fixed by hand. So the entire learning problem is the mean.

Eq. 1, 3

While training, we can cheat

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I})$$

without $\mathbf{x}_0$: intractable

with $\mathbf{x}_0$: exact formula

We can compute the perfect denoising step. Training is regression onto it.

Eq. 6, 7

Where the Π comes from

Each step only looks at the one immediately before it. So a chain is just its steps multiplied together.

$$q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) = q(\mathbf{x}_1 \mid \mathbf{x}_0) q(\mathbf{x}_2 \mid \mathbf{x}_1) \cdots q(\mathbf{x}_T \mid \mathbf{x}_{T-1}) = \prod_{t=1}^T q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

$$p_{\theta}(\mathbf{x}_{0:T}) = p_{\theta}(\mathbf{x}_T) p_{\theta}(\mathbf{x}_{T-1} \mid \mathbf{x}_T) \cdots p_{\theta}(\mathbf{x}_0 \mid \mathbf{x}_1) = p_{\theta}(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$$

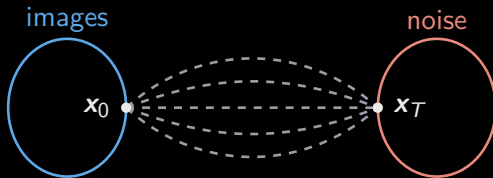
The reverse chain starts from noise, which is why $p_{\theta}(\mathbf{x}_T)$ sits out front.

The likelihood we actually want

$$q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

$$p_{\theta}(\mathbf{x}_{0:T}) = p_{\theta}(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$$

$$-\log p_{\theta}(\mathbf{x}_0) = -\log \int p_{\theta}(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T}$$



Each dashed line is one path $\mathbf{x}_0$ could have taken. The integral sums over all of them.

Turning it into a bound

$$-\log p_{\theta}(\mathbf{x}_0) = -\log \int p_{\theta}(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T}$$

marginalise over the path

$$= -\log \int q(\mathbf{x}_{1:T} | \mathbf{x}_0) \frac{p_{\theta}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} d\mathbf{x}_{1:T}$$

multiply by 1

$$= -\log \mathbb{E}_{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \left[\frac{p_{\theta}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \right]$$

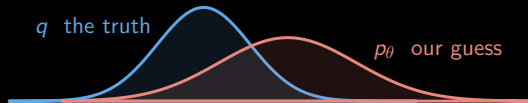
that is an expectation

$$\leq -\mathbb{E}_{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \left[\log \frac{p_{\theta}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \right]$$

Jensen

One **inequality** is the whole trick. That last line is the ELBO.

What is a KL divergence?



$$D_{\text{KL}}(q \parallel p_{\theta}) = \mathbb{E}_q \left[\log \frac{q}{p_{\theta}} \right]$$

One number for “how wrong is p_{θ} , judged against q ”.

It is 0 when they match exactly, and grows as they drift apart. It is never negative.

KL, in practice

Both of ours are Gaussians with the *same* variance σ^2 . Then the whole thing collapses:

$$D_{\text{KL}}\left(\mathcal{N}(\mu_1, \sigma^2) \parallel \mathcal{N}(\mu_2, \sigma^2)\right) = \frac{(\mu_1 - \mu_2)^2}{2\sigma^2}$$

Example: $\mu_1 = 0$, $\mu_2 = 1$, $\sigma = 1 \Rightarrow D_{\text{KL}} = 0.5$. Move the guess to $\mu_2 = 0 \Rightarrow D_{\text{KL}} = 0$.

So every KL in the ELBO is really just
a squared distance between two means.

The loss is the ELBO again

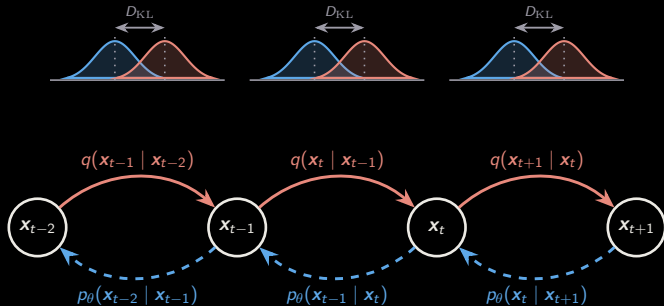
$$\mathbb{E} \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T | \mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{\text{no parameters}} + \sum_{t>1} \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \parallel p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t))}_{\text{the actual work}} - \underbrace{\log p_{\theta}(\mathbf{x}_0 | \mathbf{x}_1)}_{\text{back to pixels}} \right]$$

KL between two Gaussians = squared difference of means.

The middle term is an MSE.

Eq. 5, 8 to 12

At every step, match the posterior



Blue is the true posterior, rust is our network's guess.

Same variance in both, so closing that gap means one thing: **move the mean.**

Don't predict the mean, predict the noise

$$\mu_{\theta}(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right)$$

Everything here is known except ϵ_{θ} .

The network answers one question:
which noise was added to make you?

Eq. 11

The punchline

$$L_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \left[\left\| \epsilon - \epsilon_{\theta} \left(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t \right) \right\|^2 \right]$$

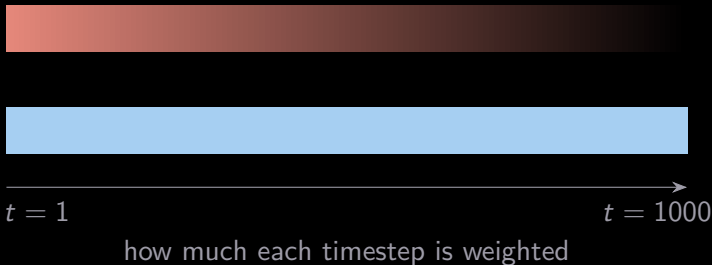
Add noise. Guess the noise. Square the error.

Eq. 14

What got quietly thrown away

true ELBO

L_{simple}



The true bound obsesses over nearly clean images.

Flat weighting spends it on **global structure** instead. A worse bound, much better pictures.

Training

repeat

$\mathbf{x}_0 \sim q(\mathbf{x}_0)$

$t \sim \text{Uniform}\{1, \dots, T\}$

$\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

step on $\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$

until converged

No discriminator. No adversarial game. One MSE.

Algorithm 1

Sampling

```
 $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
for  $t = T, \dots, 1$   
     $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$   
return  $\mathbf{x}_0$ 
```

Subtract some predicted noise, then **add a little back**.

Without that, every sample looks the same.

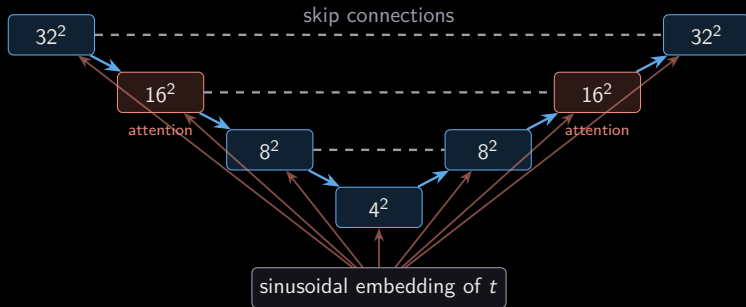
The whole training loop

```
abar = torch.cumprod(1 - torch.linspace(1e-4, 0.02, T), 0)

for x0 in loader:
    t = torch.randint(0, T, (x0.size(0),))
    eps = torch.randn_like(x0)
    xt = abar[t].sqrt()*x0 + (1-abar[t]).sqrt()*eps
    loss = F.mse_loss(model(xt, t), eps)
    loss.backward(); opt.step(); opt.zero_grad()
```

The difficulty of this paper is in the justification, not the code.

The network: our U-Net, plus a clock



Same U-Net as before. **One change:** the timestep goes into every block.

One network, every noise level



$t = 900$

what *is*
this?

same weights



$t = 100$

sharpen
this edge

t is an input, not a different model.

Coarse-to-fine generation, for free.

Results

CIFAR-10, unconditional	IS $\uparrow$	FID $\downarrow$
SNGAN	8.22	21.7
NCSNv2	8.40	10.87
StyleGAN2 + ADA	9.74	3.26
DDPM	9.46	3.17

Best FID on CIFAR-10 at the time.

Likelihood, though, is unremarkable: ≤ 3.75 bits/dim.

Optional: it's estimating a gradient

$$\epsilon_{\theta}(\mathbf{x}_t, t) \approx -\sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t)$$

Predicting noise = predicting the *score*: which way makes an image more likely.
So sampling is Langevin dynamics, and DDPM and score matching are one idea.

Section 3.2

The price

GAN   1 forward pass

DDPM   ... 1000 of them

No semantic latent to interpolate. Likelihoods lose to autoregressive models.

σ_t^2 and the β schedule are guessed by hand.

Each one became somebody's next paper.

What came next

DDIM	1000 steps $\rightarrow$ 50
Improved DDPM	learn σ_t^2 , cosine schedule
Classifier-free guidance	how a prompt steers it
Latent Diffusion	do all this inside a VAE
Flow matching	straighten the path

Every image generator you have used descends from this paper.

Five things to remember

1. A VAE whose encoder is just **noise**.
2. $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$ lets you jump anywhere in one line.
3. We rolled the noise, so we always have a label.
4. Predict the noise. Squared error. Not the true ELBO.
5. A U-Net that also reads the clock.

Thanks for watching

Next: ResNet

